



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Cloud Computing: Análisis,
Diseño y Despliegue de un
Servicio Empresarial Seguro
en Microsoft Azure
Documentación Técnica**



Presentado por Karima Drafi Rico
en Universidad de Burgos — Junio 2026
Tutor: D. José Ignacio Santos Martín

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	1
A.3. Estudio de viabilidad	2
Apéndice B Especificación de Requisitos	5
B.1. Introducción	5
B.2. Objetivos generales	5
B.3. Catálogo de requisitos	5
B.4. Especificación de requisitos	6
B.5. Casos de uso	7
Apéndice C Especificación de diseño	9
C.1. Introducción	9
C.2. Diseño arquitectónico	9
C.3. Diseño procedimental	9
C.4. Integración cloud	10
C.5. Diseño de datos	11
C.6. Calidad y observabilidad	11
Apéndice D Documentación técnica de programación	13

D.1. Introducción	13
D.2. Estructura de directorios	13
D.3. Ejecución local	14
D.4. Variables de entorno	14
D.5. Despliegue en Azure	14
D.6. Pruebas del sistema	14
D.7. Análisis de calidad	15
Apéndice E Documentación de usuario	17
E.1. Introducción	17
E.2. Enlaces principales de evaluación	17
E.3. Requisitos	18
E.4. Acceso al servicio	18
E.5. Operaciones disponibles	18
Apéndice F Anexo de sostenibilización curricular	21
F.1. Introducción	21
Bibliografía	25

Índice de figuras

Índice de tablas

A.1. Resumen de iteraciones del proyecto	2
A.2. Estimación económica del proyecto	3
A.3. Licencias y servicios utilizados	4
B.1. Casos de uso principales	8
C.1. Diccionario de datos de la solicitud	11

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este anexo describe la planificación del proyecto, el seguimiento realizado con Zube y el análisis de viabilidad del sistema.

A.2. Planificación temporal

El desarrollo del proyecto se ha organizado siguiendo un enfoque incremental basado en sprints. El seguimiento se ha realizado en Zube, utilizando historias de usuario, tareas técnicas y cambios asociados a commits del repositorio. El tablero del proyecto se encuentra en:

<https://zube.io/tfg-azure-servicio-empresarial/tfg-servicio-empresarial-seguro/w/workspace-1/kanban>

Las principales fases del proyecto han sido:

- Análisis del problema y definición de requisitos.
- Diseño de la arquitectura del sistema.
- Implementación del prototipo.
- Despliegue del sistema en la plataforma cloud.
- Validación y pruebas del sistema.

Iteraciones realizadas

La planificación se ha dividido en cinco iteraciones, superando el mínimo de tres sprints indicado en la guía docente. Cada iteración generó un incremento funcional o documental verificable:

Sprint	Objetivo	Resultado
Sprint 1	Definición del problema, requisitos iniciales y arquitectura base.	Repositorio, documentación inicial y backlog en Zube.
Sprint 2	Infraestructura Azure y persistencia cloud.	Terraform, Storage Account, Key Vault y App Service definidos.
Sprint 3	Implementación de API Flask y pruebas.	Endpoints de solicitudes, métricas, clasificación y pruebas unitarias.
Sprint 4	Despliegue, portal web y automatización.	Portal funcional en App Service, Logic App y scripts de despliegue.
Sprint 5	Cierre documental, calidad y entrega.	Memoria, anexos, bibliografía, SonarCloud preparado y evidencias de despliegue.

Tabla A.1: Resumen de iteraciones del proyecto

A.3. Estudio de viabilidad

Viabilidad económica

El proyecto se ha desarrollado utilizando principalmente herramientas disponibles en versiones gratuitas, de código abierto o académicas, como GitHub, Zube y Microsoft Azure for Students. SonarCloud se ha preparado como herramienta de análisis de calidad, pero su ejecución queda documentada como una comprobación específica previa a la entrega final.

La mayor parte del coste corresponde al esfuerzo de desarrollo. Para estimar el coste de personal se ha considerado una dedicación aproximada de 300 horas y un coste de referencia de 18 euros/hora para un perfil junior de desarrollo cloud. Sobre este coste se añade una estimación del 32% de costes empresariales asociados a Seguridad Social y otras aportaciones obligatorias. Esta cifra no representa una factura real, sino una estimación académica del valor del trabajo realizado.

En un entorno empresarial real también deberían considerarse costes de operación continuada, soporte, monitorización, almacenamiento y transfe-

Concepto	Cantidad	Coste unitario	Coste estimado
Análisis y planificación	45 h	18 EUR/h	810 EUR
Diseño de arquitectura	45 h	18 EUR/h	810 EUR
Implementación backend y portal	90 h	18 EUR/h	1.620 EUR
Infraestructura Azure y despliegue	60 h	18 EUR/h	1.080 EUR
Pruebas, documentación y memoria	60 h	18 EUR/h	1.080 EUR
Costes empresariales de personal	32 %	5.400 EUR	1.728 EUR
Servicios Azure for Students	1 suscripción	0 EUR	0 EUR
GitHub y Zube	1 cuenta académica	0 EUR	0 EUR
Total estimado	300 h		7.128 EUR

Tabla A.2: Estimación económica del proyecto

rencia de datos. En este TFG se han mantenido los recursos dentro de un entorno de desarrollo de bajo consumo, por lo que el coste de infraestructura se ha reducido al mínimo. El equipo informático y el software de escritorio no se imputan por su coste completo, ya que conservan valor tras el proyecto y pueden reutilizarse en otros trabajos; en una contabilidad empresarial se imputaría únicamente su amortización proporcional al periodo de uso.

Viabilidad legal

Durante el desarrollo del proyecto se han utilizado herramientas de software bajo licencias abiertas o gratuitas para uso académico. Además, no se han utilizado datos personales reales, por lo que no se han generado problemas relacionados con la protección de datos.

Las licencias empleadas son compatibles con un proyecto académico y permiten la distribución del código fuente del prototipo. El repositorio incluye un archivo LICENSE con licencia MIT para el código propio desarrollado

Componente	Licencia / modalidad	Uso en el proyecto
Python	PSF License	Lenguaje backend
Flask	BSD-3-Clause	API REST y portal web
Gunicorn	MIT	Servidor WSGI en App Service
Azure SDK for Python	MIT	Integración con Storage y Key Vault
Terraform	Business Source License	Infraestructura como código
GitHub Actions	Servicio GitHub	Integración continua y despliegue
SonarCloud	Servicio SonarSource	Análisis de calidad preparado para la entrega
Microsoft Azure	Servicio cloud	App Service, Storage, Key Vault y Logic App

Tabla A.3: Licencias y servicios utilizados

durante el TFG. Las credenciales y secretos no se almacenan en el repositorio, sino en variables de entorno, GitHub Secrets y Azure Key Vault.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este anexo describe los requisitos funcionales y no funcionales definidos para el sistema desarrollado.

B.2. Objetivos generales

El objetivo del sistema es permitir la gestión segura de solicitudes operativas TI utilizando servicios cloud desplegados en Microsoft Azure. Una incidencia se considera un tipo concreto de solicitud, junto con accesos, cambios de configuración, altas de aplicaciones o creación de entornos.

B.3. Catálogo de requisitos

Los requisitos funcionales recogen las capacidades visibles del sistema para usuarios, API y automatizaciones externas:

Requisitos funcionales

- RF-01 Crear solicitudes operativas TI.
- RF-02 Consultar solicitudes.
- RF-03 Obtener métricas del sistema.
- RF-04 Gestionar almacenamiento cloud.

- RF-05 Gestionar secretos mediante Azure Key Vault.
- RF-06 Clasificar solicitudes automáticamente.

Los requisitos no funcionales recogen propiedades transversales necesarias para que el sistema sea evaluable, seguro y mantenible:

Requisitos no funcionales

- RNF-01 Seguridad.
- RNF-02 Escalabilidad.
- RNF-03 Disponibilidad.
- RNF-04 Trazabilidad.

B.4. Especificación de requisitos

RF-01 Crear solicitudes operativas TI

El sistema debe permitir registrar solicitudes mediante `POST /solicitudes`, validando título, descripción, tipo de solicitud y email del solicitante. El endpoint `POST /incidencias` se mantiene como compatibilidad para el caso específico de incidencias.

RF-02 Consultar solicitudes

El sistema debe listar solicitudes mediante `GET /solicitudes`, con filtros opcionales por estado, prioridad y tipo. Esta operación requiere autenticación mediante token Bearer.

RF-03 Obtener métricas

El sistema debe exponer métricas agregadas en `GET /metrics`.

RF-04 Gestionar almacenamiento cloud

Las solicitudes deben persistirse en Azure Blob Storage en entorno cloud y en fichero JSON en desarrollo local.

RF-05 Gestionar secretos mediante Azure Key Vault

El token de autenticación debe almacenarse en Key Vault y recuperarse mediante Managed Identity.

RF-06 Clasificación automática

El sistema debe clasificar automáticamente prioridad, categoría y tipo de cada solicitud, generando además una recomendación operativa.

RNF-01 Seguridad

Autenticación Bearer, almacenamiento privado y gestión de secretos centralizada.

RNF-02 Escalabilidad

Arquitectura basada en servicios gestionados de Azure App Service y Blob Storage.

RNF-03 Disponibilidad

Endpoint de salud /health y despliegue automatizado.

RNF-04 Trazabilidad

Registro de fecha de creación, estado, prioridad, tipo, clasificación y recomendación de cada solicitud.

B.5. Casos de uso

El sistema se ha diseñado alrededor de los siguientes casos de uso principales:

Caso de uso	Actor	Descripción
CU-01 Registrar solicitud	Usuario interno	Envía una solicitud desde el portal o mediante API REST.
CU-02 Clasificar solicitud	Sistema	Asigna tipo, prioridad, categoría y recomendación operativa.
CU-03 Consultar solicitudes	Equipo TI	Lista solicitudes y filtra por estado, tipo o prioridad.
CU-04 Consultar métricas	Equipo TI	Revisa métricas agregadas para seguimiento operativo.
CU-05 Automatizar registro	Sistema externo	Usa Logic App para iniciar el flujo sin acceder directamente a la API.
CU-06 Gestionar secretos	Administrador cloud	Mantiene tokens y configuración sensible en Azure Key Vault.

Tabla B.1: Casos de uso principales

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo describe el diseño general de la plataforma cloud para la automatización de solicitudes operativas TI.

C.2. Diseño arquitectónico

La arquitectura sigue un modelo basado en servicios cloud desplegados sobre Microsoft Azure. La aplicación se ejecuta en Azure App Service, utiliza Azure Blob Storage para persistir solicitudes, Azure Key Vault para secretos y Managed Identity para reducir la exposición de credenciales [7, 9, 8, 10].

Los principales componentes son:

- Azure App Service (API Flask y portal web).
- Azure Storage (persistencia de solicitudes).
- Azure Key Vault (gestión de secretos).
- Azure Logic App (orquestración del flujo).
- GitHub Actions (integración y despliegue continuo).

C.3. Diseño procedimental

El flujo principal del sistema es:

1. Recepción de solicitudes operativas TI por portal web, API REST o Logic App.
2. Validación de campos obligatorios.
3. Clasificación automática mediante componente de IA ligera.
4. Generación de recomendaciones de actuación operativa.
5. Persistencia de solicitudes en Blob Storage o fichero local.
6. Respuesta JSON con identificador, prioridad, tipo, recomendación y confianza.
7. Automatización del flujo mediante Logic App y notificación al equipo TI.

Tipos de solicitud soportados

- Acceso a recursos (VPN, permisos, cuentas).
- Provisión de entornos (desarrollo, staging, producción).
- Alta de aplicaciones o servicios.
- Cambios de configuración.
- Incidencias o peticiones de soporte.

C.4. Integración cloud

- **Terraform**: provisiona Resource Group, App Service, Storage y Key Vault.
- **Bicep**: plantillas alternativas para reproducir la infraestructura.
- **GitHub Actions**: prueba, valida y despliega la solución.
- **Logic App**: automatiza el procesamiento de solicitudes desde un trigger HTTP.

C.5. Diseño de datos

La persistencia del prototipo utiliza un documento JSON por solicitud en Azure Blob Storage. No existe una base de datos relacional, por lo que el modelo de datos se describe como una entidad documental.

Campo	Tipo	Descripción
id	texto	Identificador único de la solicitud.
titulo	texto	Resumen de la petición enviada por el usuario.
descripcion	texto	Detalle de la solicitud o incidencia.
tipo_solicitud	texto	Acceso, entorno, aplicación, configuración o incidencia.
prioridad	texto	Prioridad calculada por el clasificador.
categoria	texto	Área funcional detectada: infraestructura, seguridad, software u otra.
estado	texto	Estado operativo de la solicitud.
recomendacion	texto	Acción sugerida para el equipo TI.
reportado_por	texto	Correo o identificador del solicitante.
created_at	fecha	Fecha de creación de la solicitud.

Tabla C.1: Diccionario de datos de la solicitud

C.6. Calidad y observabilidad

La calidad se comprueba mediante pruebas automáticas y un flujo de análisis preparado con SonarCloud. En el momento de cerrar este anexo, SonarCloud está configurado en el repositorio mediante `sonar-project.properties` y el workflow `SonarCloud Quality Analysis`, pero debe ejecutarse antes de la entrega para obtener la URL y la captura final del panel de calidad. La observabilidad técnica se apoya en el endpoint `/health`, en los logs de App Service y en las métricas disponibles desde Azure Monitor [6].

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este anexo describe la estructura técnica del repositorio y los pasos para ejecutar, probar y desplegar la solución, apoyándose en despliegue continuo con GitHub Actions y servicios gestionados de Azure [2, 5, 10].

D.2. Estructura de directorios

- `src/app.py`: API principal Flask.
- `src/classifier.py`: clasificador automático de incidencias.
- `src/storage.py`: persistencia local o en Azure Blob Storage.
- `src/config.py`: configuración por variables de entorno.
- `src/requirements.txt`: dependencias Python.
- `tests/test_api.py`: pruebas unitarias.
- `infra/terraform/`: infraestructura como código.
- `logicapp/`: definición de Logic App.
- `.github/workflows/`: CI/CD con GitHub Actions.
- `memoria/tex/`: memoria y anexos en LaTeX.

D.3. Ejecución local

```
cd src
pip install -r requirements.txt
set STORAGE_MODE=local
python app.py
```

La API quedará disponible en `http://localhost:5000`.

D.4. Variables de entorno

- `STORAGE_MODE`: `local` o `azure`.
- `AZURE_STORAGE_CONNECTION_STRING`: cadena de conexión del Storage Account.
- `KEY_VAULT_URL`: URI del Key Vault.
- `API_KEY`: token Bearer opcional para desarrollo local.

D.5. Despliegue en Azure

1. Configurar el secreto `AZURE_CREDENTIALS` en GitHub.
2. Ejecutar `terraform apply` en `infra/terraform`.
3. Desplegar la carpeta `src` en App Service mediante GitHub Actions o mediante el script `scripts/deploy-azure.ps1`.
4. Importar la Logic App desde `logicapp/logicapp-workflow.json`.

D.6. Pruebas del sistema

```
python -m unittest discover -s tests -p "test_*.py"
```

Las pruebas verifican endpoints, creación de incidencias, métricas y clasificación automática.

D.7. Análisis de calidad

El repositorio incluye la configuración necesaria para ejecutar SonarCloud como servicio gestionado de SonarQube:

- `sonar-project.properties`: define clave de proyecto, organización, fuentes y tests.
- `.github/workflows/sonarcloud.yml`: instala dependencias, ejecuta pruebas y lanza el análisis.
- `docs/calidad/sonarcloud.md`: documenta la puesta en marcha y las evidencias que deben guardarse.

Para completar la evidencia de calidad se debe crear el secreto `SONAR_TOKEN` en GitHub Actions y ejecutar el workflow `SonarCloud Quality Analysis`. La URL del panel de calidad y una captura del Quality Gate deben incorporarse al documento final de enlaces o a la documentación de evidencias.

Apéndice *E*

Documentación de usuario

E.1. Introducción

Este anexo describe cómo utilizar la plataforma de gestión de solicitudes operativas TI desplegada en Microsoft Azure.

E.2. Enlaces principales de evaluación

Los siguientes enlaces permiten acceder directamente a los recursos principales que debe revisar el tribunal:

- **Portal web desplegado**
<https://app-tfg-incidencias-dev.azurewebsites.net/portal>
- **Repositorio GitHub**
<https://github.com/karimadrico/tfg-azure-servicio-empresarial-seguro>
- **Tablero Zube utilizado para la planificación por sprints**
<https://zube.io/tfg-azure-servicio-empresarial/tfg-servicio-empresarial-seguro/w/workspace-1/kanban>
- **Panel SonarCloud utilizado para el seguimiento de calidad del código**
https://sonarcloud.io/project/overview?id=karimadrico_tfg-azure-servicio-empresarial-seguro

E.3. Requisitos

El usuario necesita:

- Cliente HTTP (navegador, Postman o curl).
- Conexión a Internet.
- Token de autenticación Bearer (excepto en entorno local sin clave configurada).

E.4. Acceso al servicio

La API está desplegada en Azure App Service:

`https://app-tfg-incidencias-dev.azurewebsites.net`

El portal web está disponible en:

`https://app-tfg-incidencias-dev.azurewebsites.net/portal`

También puede registrarse una solicitud mediante la Logic App, que reenvía la petición automáticamente a la API.

E.5. Operaciones disponibles

Consultar estado del servicio

GET /

Crear solicitud desde el portal

El usuario puede abrir el portal web, seleccionar el tipo de solicitud, introducir título, descripción y correo del solicitante, y enviar el formulario. La API devolverá un identificador, prioridad, clasificación y recomendación operativa.

Crear solicitud desde API

POST /solicitudes

Content-Type: application/json

```
{
  "tipo_solicitud": "acceso",
  "titulo": "Acceso VPN",
  "descripcion": "Necesito acceso VPN al entorno cloud",
  "reportado_por": "usuario@empresa.com",
  "prioridad": "media"
}
```

El sistema clasificará automáticamente la solicitud y devolverá prioridad, categoría, tipo, nivel de confianza y recomendación.

Consultar solicitudes

GET /solicitudes?estado=abierta&prioridad=alta&tipo_solicitud=acceso

Authorization: Bearer <token>

Si se accede a esta URL directamente desde el navegador sin cabecera de autorización, la API responde `{.error:"No autenticado"}`, que es el comportamiento esperado para proteger el listado.

Consultar métricas

GET /metricas

Authorization: Bearer <token>

Anexo de sostenibilización curricular

F.1. Introducción

El desarrollo de soluciones cloud puede contribuir significativamente a mejorar la sostenibilidad tecnológica y organizativa cuando se diseña con criterios de eficiencia, seguridad y responsabilidad [1, 3]. En este proyecto se ha desarrollado una plataforma para automatizar solicitudes operativas TI en Microsoft Azure, evitando un enfoque basado en procesos manuales, correos dispersos o documentación no estructurada. Esta orientación se relaciona con varios Objetivos de Desarrollo Sostenible, especialmente con el ODS 9, industria, innovación e infraestructura; el ODS 12, producción y consumo responsables; y el ODS 13, acción por el clima.

Desde el punto de vista del ODS 9, la solución propone una infraestructura digital moderna basada en servicios gestionados. El uso de Azure App Service, Storage, Key Vault y Logic Apps permite construir un sistema escalable sin necesidad de adquirir, instalar y mantener servidores físicos propios. La infraestructura como código mediante Terraform y Bicep favorece la reproducibilidad, reduce errores de configuración y permite que el entorno pueda recrearse de forma controlada, en línea con buenas prácticas de arquitectura cloud [4]. Esto contribuye a una innovación más sostenible, ya que el conocimiento generado no queda limitado a una configuración manual difícil de auditar.

En relación con el ODS 12, el proyecto fomenta un uso responsable de recursos tecnológicos. La aplicación se ha desplegado en un entorno

de bajo consumo, ajustado al alcance académico y sin sobredimensionar la infraestructura. La plataforma cloud permite activar únicamente los recursos necesarios para validar el prototipo, evitando gastos innecesarios y reduciendo el desperdicio tecnológico. Además, la automatización de despliegues y verificaciones disminuye la repetición de tareas manuales y reduce la probabilidad de errores humanos, lo que mejora la eficiencia operativa.

El ODS 13 también está presente de forma indirecta. Aunque cualquier servicio digital consume energía, el uso de proveedores cloud permite beneficiarse de centros de datos optimizados, con mejores tasas de utilización que muchas infraestructuras locales pequeñas. En lugar de mantener servidores dedicados infrautilizados, el proyecto usa servicios compartidos y escalables. Esta decisión no elimina el impacto ambiental, pero sí representa una alternativa más eficiente para un prototipo académico y para muchas organizaciones pequeñas que necesitan digitalizar procesos internos.

La sostenibilidad también tiene una dimensión social y organizativa. La plataforma mejora la trazabilidad de solicitudes internas, facilita priorizar incidencias críticas, registra recomendaciones operativas y permite consultar métricas del servicio. Esto puede ayudar a equipos técnicos a trabajar con mayor transparencia y a reducir tiempos de respuesta. Un proceso más ordenado también evita duplicidades, pérdidas de información y dependencia excesiva de conocimiento informal.

Por último, el proyecto incorpora criterios de seguridad y protección de la información. El uso de Key Vault, Managed Identity y autenticación Bearer evita almacenar secretos directamente en el código fuente, siguiendo recomendaciones de diseño seguro en Azure [11]. Esta práctica favorece un desarrollo responsable y reduce riesgos asociados a fugas de credenciales. Aunque el sistema utiliza datos de prueba y no datos personales reales, se han aplicado principios básicos de minimización y protección.

Otra aportación relevante es la mantenibilidad del sistema. La solución se documenta mediante memoria, anexos, guías de despliegue y scripts reproducibles, de forma que otra persona pueda revisar el diseño, ejecutar pruebas y desplegar el prototipo sin depender únicamente de conocimiento verbal. Esta trazabilidad técnica reduce el riesgo de abandono del software tras la entrega académica y facilita que el trabajo pueda evolucionar hacia nuevas funciones, como autenticación corporativa, mejoras de observabilidad o integración con más flujos empresariales.

En conjunto, el TFG demuestra que una solución cloud académica puede diseñarse no solo para funcionar, sino también para ser mantenible, segura,

eficiente y alineada con criterios de sostenibilidad curricular. La principal aportación no es únicamente el software desarrollado, sino la aplicación de buenas prácticas profesionales para automatizar procesos internos con un impacto razonable y controlado.

Bibliografía

- [1] CRUE Universidades Españolas. Directrices de sostenibilidad en trabajos de fin de grado y proyectos, 2012. Consultado: marzo 2026.
- [2] GitHub. Github actions documentation, 2024. Consultado: marzo 2026.
- [3] Microsoft Azure. Microsoft cloud adoption framework for azure, 2024. Guía oficial de buenas prácticas de adopción de nube de Microsoft Learn para entornos Azure, consultado marzo 2026.
- [4] Microsoft Azure. Azure well-architected framework, 2025. Marco de mejores prácticas para diseño, fiabilidad, seguridad y costes en arquitecturas cloud de Azure, consultado marzo 2026.
- [5] Microsoft Learn. Azure key vault documentation, 2024. Consultado: marzo 2026.
- [6] Microsoft Learn. Azure monitor documentation, 2024. Consultado: marzo 2026.
- [7] Microsoft Learn. Configure a linux python app for azure app service, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [8] Microsoft Learn. Managed identities for azure app service, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [9] Microsoft Learn. Quickstart: Azure blob storage client library for python, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.

- [10] Microsoft Learn. Use key vault references as app settings in azure app service, azure functions, and azure logic apps, 2026. Documentación oficial de Microsoft Learn, consultado junio 2026.
- [11] Microsoft Learn. Área de diseño: Seguridad - cloud adoption framework, 2026. Guía de diseño de seguridad en Azure (Cloud Adoption Framework), consultado marzo 2026.